



An Axiomatic Concurrency Model

4

5

```
def update(account: cown[Account], amount: int) {  
  when(account) {  A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) {  B  
        log.record(amount)  
      }  
    }  
  }  
}
```



S



—

R  +

R



-



S



—



C



-

R  +

R.B. -



C



—





po

po







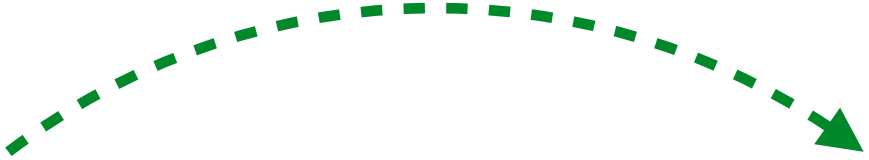
PO

po

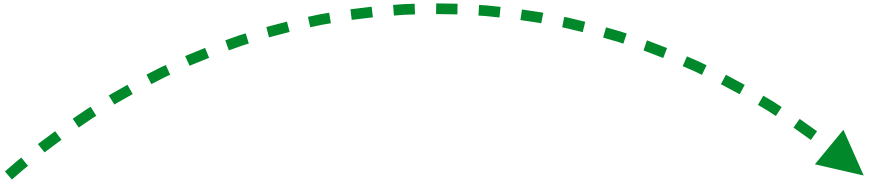


po

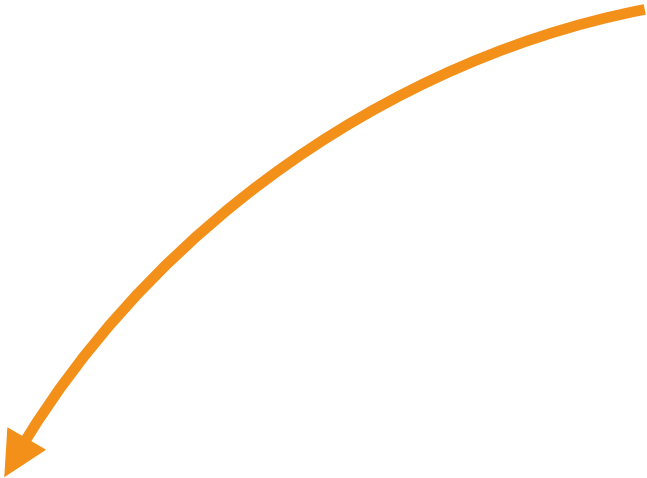
po



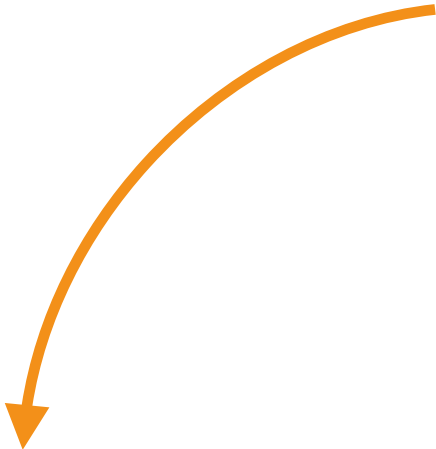








cosrc



collog

We derive the run relation from behaviour to run

```
update(src, +100)
```



```
update(src, -100)
```

Candidate executions are tuples of (E, po, co)

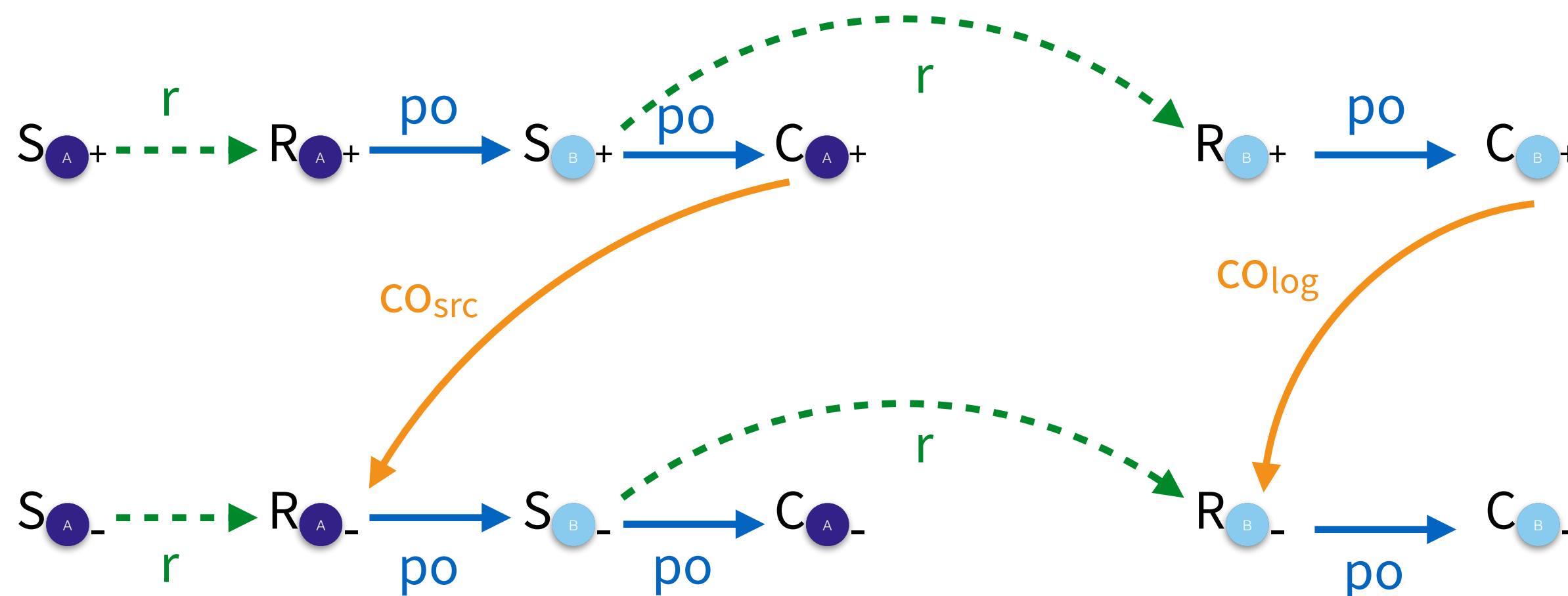
An Axiomatic Concurrency Model

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B  
        log.record(amount)  
      }  
    }  
  }  
}
```

update(src, +100)

update(src, -100)

Candidate executions are tuples of $(\mathbb{E}, po, co)$



We derive the run relation from behaviour spawn to run

An Axiomatic Concurrency Model

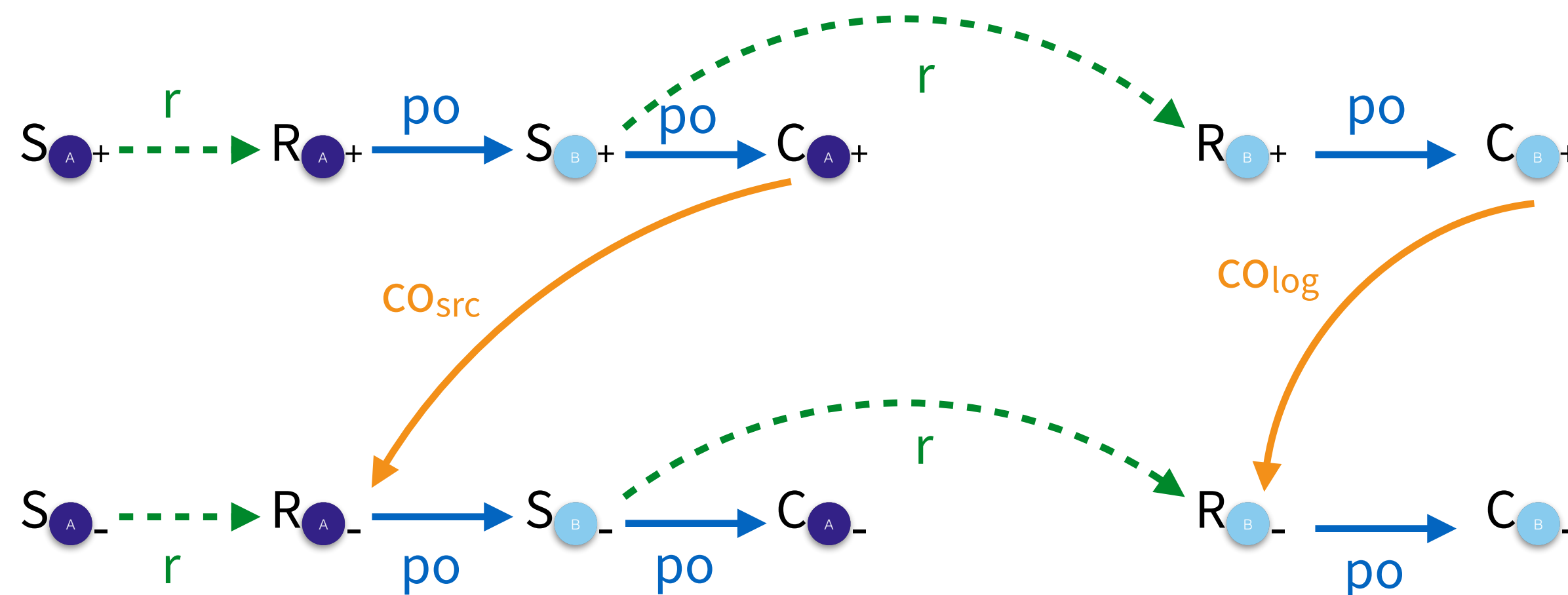
```

def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B
        log.record(amount)
      }
    }
  }
}

```

update(src, +100)

update(src, -100)



We derive happens before

$$hb = \{(C_i, R_j) \mid S_i (po \cup r \cup co)^+ S_j \wedge \text{cowns}(i) \cap \text{cowns}(j) \neq \emptyset \dots\}$$

Is there a causal path between the spawns of two behaviours that require the same cowns